



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CAMPUS NATAL CENTRAL
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

DIM0549 - GRAFOS : ORIENTEERING PROBLEM (OP)

HEITOR FERNANDES PAES LEME CAMPOS
LEANDRO ANDRADE DE ARAUJO

Natal/RN - 2026

Sumário

1. Ciclo Evolutivo Principal-----	2
2. Estratégias de Seleção e Sobrevivência-----	2
3. Operador de Recombinação Genética-----	3
4. Otimização e Algoritmo Memético-----	3
5. Controle de Estagnação-----	4
6. Heurísticas de Construção e Inicialização-----	4
7. Tabela de Comparações do GA-----	5
8. Conclusão-----	6

Para a resolução do Problema de Orientação (OP), a arquitetura do algoritmo foi dividida em componentes modulares que gerenciam a evolução da população, a seleção de indivíduos, a recombinação genética e a otimização através de busca local. Abaixo, detalham-se as principais funções e operadores utilizados:

1. Ciclo Evolutivo Principal

- **evolve**: Atua como o núcleo de controle do Algoritmo Genético. Esta função gerencia a geração, integrando a seleção de pais, o cruzamento genético, a mutação, o cálculo *fitness* e a substituição populacional. O seu *design* modular permite desacoplar a lógica principal das heurísticas específicas, garantindo flexibilidade para testar diferentes estratégias sem alterar a base do sistema.
- **calc_score**: Percorre a rota do indivíduo para acumular a distância gasta e os lucros recolhidos. A função também possui um tratamento de violações do limite de custo **TMax**. Em vez de descartar rotas impossíveis, a função aplica uma penalidade matemática proporcional ao excesso de distância diretamente no *fitness*

$$fitness = score / (1.0 + (distância - TMax) * penaltyDelta * 5.0)$$

Isso permite ao algoritmo explorar a fronteira entre regiões possíveis e impossíveis, preservando rotas promissoras que necessitam apenas de pequenos ajustes topológicos para se tornarem válidos. Além disso, inspirado pelo artigo “**A Genetic Algorithm for the Orienteering Problem**”, utilizamos um parâmetro *penaltyDelta* para evitar que gerações possuam filhos impróprios para o problema, fazendo com que eles percam a prioridade, evitando a reprodução e seleção deles.

2. Estratégias de Seleção e Sobrevivência

A gestão da diversidade e da pressão evolutiva é realizada através de três funções principais:

- **tournament (Seleção por Torneio)**: Utilizada para a seleção de pais, esta técnica agrupa aleatoriamente um subconjunto de indivíduos e escolhe o de maior *fitness*. Sua vantagem é a capacidade de controlar a pressão de seleção: torneios mantêm um bom equilíbrio entre a seleção dos melhores indivíduos e a chance de reprodução de indivíduos medianos, preservando a diversidade genética.
- **random_choice (Seleção Aleatória)**: Escolhe indivíduos de forma puramente aleatória através de uma distribuição uniforme. É utilizada em conjunto com outras estratégias para promover exploração profunda, garantindo que material genético atípico não seja sumariamente descartado nas primeiras gerações.
- **steadyState (Substituição Populacional)**: Implementa o modelo de Estado Estacionário, garantindo a natureza elitista da evolução. A função compara os novos filhos gerados com os piores indivíduos da geração anterior; a substituição só ocorre se os filhos apresentarem maior aptidão. Este método evita a regressão evolutiva, assegurando que as melhores soluções encontradas não sejam perdidas.

3. Operador de Recombinação Genética

- **x1 (Crossover Variável):** Desenvolvido especificamente para contornar a limitação de rotas de tamanhos variáveis no OP (inspirada no crossover do artigo enviado). A função herda um segmento contínuo de nós do "pai" e preenche os espaços restantes com material da "mãe", aplicando inserções aleatórias caso a rota não alcance o tamanho médio estimado. Este operador permite combinar blocos construtivos altamente lucrativos de ambos os pais.

4. Otimização e Algoritmo Memético

O refinamento das soluções encontradas pelo Algoritmo Genético é feito pelo componente memético.

- **memeticMut (Mutaç o Mem tica):** Aplica muta  es estruturais (com probabilidade) e testa iterativamente diversos operadores de inser  o, remo  o e substitui  o. Calcula o *fitness* de cada perturba  o e aplica apenas a melhor altera  o de forma gulosa. Isso permite que a solu  o convirja rapidamente para  timos locais de alta qualidade.
- **intensifyBest (Refinamento Fino):** Localiza o melhor indiv duo da popula  o e aplica a fun  o **memeticMut** repetidas vezes at  que haja estagna  o (nenhuma melhoria de *fitness* detectada). Este mecanismo garante a visita das regi  es mais promissoras do espa o de busca.

Operadores Internos da Busca Local:

- **omit_a_point:** Remove um n  aleat rio da rota. Fundamental para liberar "or amento de dist ncia", abrindo espa o para futuras inser  es de cidades mais valiosas. (inspirado pelo artigo enviado)
- **add_a_point:** Insere um n  ausente na posi  o da rota que resulte no menor incremento poss vel de dist ncia, respeitando o **TMax**.   o principal motor para aumentar o lucro. (inspirado pelo artigo enviado)
- **replace_a_point:** Substitui um n  pertencente   rota por outro ausente, com base numa heur stica de custo-benef cio. Permite trocas inteligentes de cidades geograficamente pr ximas, mas com lucros distintos. (inspirado pelo artigo enviado)
- **shift :** Realoca um n  para uma posi  o diferente na mesma rota, visando reduzir a dist ncia total percorrida e desfazer inefici ncias.
- **swap:** Permuta as posi  es de dois n s presentes na rota, corrigindo cruzamentos ineficientes.
- **invert :** Inverte a ordem de visita  o de um segmento cont nuo da rota, visando desembara ar arestas e encontrar poss veis caminhos melhores.

5. Controle de Estagnação

- **immigrate**: Para contornar problemas de falta de variabilidade genética, este operador substitui uma parcela dos piores indivíduos por novas soluções puramente aleatórias.

6. Heurísticas de Construção e Inicialização

Para que o Algoritmo Genético tenha um ponto de partida robusto e diversificado, a geração da população inicial não deve depender de um único método. Para isso, o sistema utiliza três abordagens distintas para construir rotas a partir do zero, balanceando a qualidade inicial da solução e a diversidade genética.

- **generate**: Constrói uma rota partindo do nó inicial e, a cada passo, seleciona o próximo nó de forma puramente aleatória dentre as cidades ainda não visitadas, parando apenas quando a adição de um novo nó ultrapassaria o limite de tempo/distância **TMax**. Geralmente as soluções encontradas possuem fitness baixo, embora tenha alta variedade genética.
- **genNearestNeighbor**: É uma abordagem construtiva gulosa que a partir do nó atual, o algoritmo avalia todas as cidades não visitadas e escolhe iterativamente aquela que possui a menor distância geográfica para ser o próximo destino, desde que a viagem para essa cidade (e para o nó final) não viole o orçamento **TMax**. Geralmente as rotas são bem otimizadas no quesito *fitness* e fornece uma boa base para o começo da população.
- **genNearestInsertion**: Esta heurística começa com uma rota básica (apenas o nó inicial e final) e avalia todas as cidades não visitadas. Ela seleciona a cidade que oferece a melhor relação custo-benefício (menor desvio de distância extra gerado pela sua inserção ou a melhor razão lucro/desvio) e a insere na melhor posição possível da rota. Geralmente as rotas são bem otimizadas no quesito *score*, o que acelera o processo de amadurecimento da população.

7. Tabela de Comparações do GA

Tmax	GA Artigo	GA All	MA all	GA GNI	MA GNI	GA GNN	MA GNN	GA GG	MA GG
5	10	10	10	10	10	10	10	10	10
10	15	15	15	15	15	15	15	15	15
20	65	65	65	65	65	65	65	65	65
30	115	110	110	110	110	110	110	110	110
40	155	150	150	150	150	150	150	150	150
50	190	190	190	190	190	190	190	190	190
60	225	220	220	220	220	220	220	220	220
70	260	260	260	260	260	260	260	260	260
80	280	280	280	280	280	280	280	280	280
85	285	285	285	285	285	285	285	285	285

Tabela de comparação de score de indivíduo

TMax	D artigo	D encontrada	Rota
5	4.14	4.14	1 28 32
10	6.87	9.90	1 28 19 32
20	14.70	19.85	1 28 27 31 26 20 21 19 32
30	29.94	29.94	1 13 9 8 10 11 12 21 22 26 31 27 28 32
40	38.97	39.91	1 28 27 31 26 25 23 22 21 12 11 10 8 2 3 7 13 32
50	49.19	49.79	1 28 27 31 26 25 24 23 22 21 12 11 10 8 2 3 7 4 5 6 32
60	59.89	59.87	1 19 20 27 31 30 26 25 24 23 22 21 12 11 10 8 2 3 7 6 4 5 18 28 32
70	69.56	69.96	1 28 29 17 16 15 14 4 5 6 7 3 2 8 9 10 11 12 21 22 23 24 25 26 31 27 19 32
80	79.30	79.88	1 28 29 17 16 15 14 4 5 6 7 3 2 8 10 11 12 21 22 23 24 25 26 30 31 27 20 19 9 13 32
85	82.89	84.89	1 29 17 16 15 14 4 5 6 13 7 3 2 8 9 10 11 12 21 22 23 24 25 30 31 26 27 20 19 28 18 32

Tabela de comparação de lucros de indivíduo

8. Conclusão

Portanto, apesar de não obtermos scores que superem completamente os artigos de base, obtivemos resultados positivos considerando que boa parte das rotas já estavam maximizadas. Além disso, conseguimos analisar como a mudança de cidades pode afetar o custo da rota indiretamente, possibilitando a manipulação de novas cidades na rota. Por fim, acreditamos que a mudança de hiperparâmetros pode melhorar o score final e a consistência do algoritmo, permitindo testes futuros utilizando a estratégia abordada.